

LangChain必背内容

一、必背内容（核心知识点）

1. 核心 API: create_agent

- **唯一入口**：v1.x 所有 Agent 必须通过 `from langchain.agents import create_agent` 创建。
- **必填参数**：`model="provider:model_name"`。
- **可选核心参数**：`tools`、`system_prompt`、`checkpointers`、`middleware`、`response_format`、`interrupt_before/after`。
- **调用方式**：
 - `agent.invoke({"messages": [...]}, config=...)` 返回完整结果字典。
 - 最终回答通过 `result["messages"][-1].content` 获取。

2. 消息流转机制（Message Types）

类型	关键点
SystemMessage	系统级指令，最高优先级
HumanMessage	用户输入
AIMessage	模型回复，可包含 <code>tool_calls</code> （列表）
ToolMessage	工具返回结果，必须带 <code>tool_call_id</code> 对应 <code>AIMessage</code> 中的 <code>id</code>
AIMessageChunk	流式输出片段

核心规则：工具调用后的 `ToolMessage` 必须紧跟在含有相应 `tool_calls` 的 `AIMessage` 之后，否则模型会报错。

3. 工具定义三种方式

方式	适用场景	关键点
----	------	-----

纯函数	简单逻辑	要写完整的类型注解和 docstring，LLM 通过 docstring 理解工具用途
@tool 装饰器	需配置 response_format	支持 response_format="content_and_artifact"
@tool(args_schema=...)	复杂嵌套参数	配合 Pydantic BaseModel 定义输入结构

最佳实践：工具始终返回字符串，异常必须 try/except，docstring 要写明触发条件。

4. 中间件系统 (Middleware)

- **洋葱模型：**请求按顺序经过中间件的 before 钩子，核心 Agent 执行，然后逆序经过 after 钩子。
- **6 个生命周期钩子：** before_agent, before_model, wrap_model_call, after_model, wrap_tool_call, after_agent。
- **十大内置中间件：**
 - SummarizationMiddleware：对话摘要，参数 trigger（消息数/token/比例）和 keep。
 - PIIMiddleware：脱敏，支持 mask/redact/block/hash 策略，可自定义 detector。
 - ModelCallLimitMiddleware：防无限循环（run_limit, thread_limit）。
 - ModelFallbackMiddleware：模型降级列表。
 - HumanInTheLoopMiddleware：人工审批，需配合 checkpointer。
 - ToolRetryMiddleware：工具重试（指数退避）。
 - ContextEditingMiddleware：自动裁剪旧工具调用结果。
 - LLMToolSelectorMiddleware：用小模型筛选工具。
- **自定义中间件：**支持 @dynamic_prompt 动态修改 system prompt，@wrap_model_call 动态切换模型等。

5. 记忆与多轮对话

- **无状态：**Agent 本身不记忆，必须通过 Checkpointer 或手动维护消息列表。
- **三种策略：**

策略	原理	适用场景
全量记忆	保存全部历史消息	短对话
摘要记忆	超阈值时 LLM 总结	常规客服（推荐）
向量记忆	Embedding + 向量检索	长期跨会话

• Checkpointer 使用：

Python

```
1 from langgraph.checkpoint.memory import MemorySaver
2 agent = create_agent(checkpointer=MemorySaver())
3 config = {"configurable": {"thread_id": "user_001"}}
```

持久化可用 `SqliteSaver.from_conn_string("chat.db")`。

6. 多 Agent 协作（Supervisor 模式）

- 三层架构：底层 API 工具 → 中层专业 Agent（自然语言→结构化）→ 顶层 Supervisor 路由。
- 关键做法：将子 Agent 包装为 `@tool`，Supervisor 无需感知其内部逻辑。
- 调用隔离：子 Agent 的 `invoke` 使用独立的 `messages` 列表，不与 Supervisor 混淆。

7. 结构化输出与调试

- `response_format=PydanticModel`：强制模型输出 JSON 格式，自动校验。
- 调试技巧：遍历 `result["messages"]`，检查 `tool_calls` 和内容。

二、面试高频问题

1. `create_agent` 和旧版 `AgentExecutor` / `initialize_agent` 有什么区别？

- 答案：`create_agent` 是 v1.x 唯一入口，底层基于 LangGraph，支持 Middleware、Checkpointer、结构化输出等；旧版链式 API 已废弃。统一了 Agent 构建方式。

2. Agent 如何决定是否调用工具？（ReAct 循环机制）

- 答案：Agent 根据用户输入和消息历史调用 LLM，LLM 返回的 `AIMessage` 中如果包含 `tool_calls`，Agent 就会执行对应工具，然后将 `ToolMessage` 追加回列表，再次调用 LLM，直到 LLM 返回纯文本回复为止。这就是 ReAct（Reasoning + Acting）循环。

3. 为什么多轮对话中必须用 `result["messages"]` 覆盖本地列表？

- **答案：**因为中间过程会产生 `AIMessage(tool_calls=...)` 和 `ToolMessage` 等“隐形消息”，如果只追加最终的 `content`，会丢失这些必要信息，下次请求时模型会因找不到对应的 `ToolMessage` 而报错。

4. 中间件 Middleware 的作用？请举出至少三个内置中间件的使用场景。

- **答案：**在不修改业务代码的情况下横切注入逻辑。例如：
 - `PIIMiddleware`：脱敏用户手机号，防止泄露。
 - `SummarizationMiddleware`：长对话自动摘要，控制 Token 消耗。
 - `ModelCallLimitMiddleware`：防止工具误调用导致无限循环。
 - `HumanInTheLoopMiddleware`：敏感操作（如发邮件）需人工审批。

5. 如何实现持久化多轮对话？

- **答案：**使用 `Checkpointter`，如 `MemorySaver`（内存）或 `SqliteSaver`（磁盘）。只需在 `create_agent` 中传入 `checkpointer`，每次 `invoke` 时传入同一个 `thread_id`，框架自动存储和恢复历史。

6. 什么是 Supervisor 多 Agent 架构？它解决了什么问题？

- **答案：**将复杂任务拆分为多个专业子 Agent（如日历、邮件、搜索），再通过一个主管 Agent 统一路由。解决了单个 Agent 能力边界和上下文管理问题。子 Agent 作为工具暴露给 Supervisor，形成三层抽象（工具→子Agent→Supervisor）。

7. 工具定义时为什么必须写详细的 Docstring？

- **答案：**LLM 通过 `docstring` 理解工具的功能、触发条件和参数含义。清晰准确的 `docstring` 直接影响工具被正确调用的概率。文档建议包含：工具概述、触发场景、每个参数说明。

8. 请解释 Content Blocks 的作用。

- **答案：**`v1.x` 用统一的 `content_blocks` 规范不同模型返回的异构格式（如 OpenAI 的 `function_call`、Claude 的 `tool_use`），开发者只需通过 `block.type` 判断文本/推理/工具调用/引用，无需再处理各厂商的原始格式。

9. 如何处理 Agent 的主动中断（Human-in-the-Loop）？

- **答案：**设置 `HumanInTheLoopMiddleware` 并指定需中断的工具名，同时在 `create_agent` 中提供 `checkpointer`。当 Agent 尝试调用该工具时，会暂停并返回待审批的中间状态。调用方检查 `last_msg.tool_calls`，经人工同意后通过 `agent.invoke(Command(resume=ApproveDecision()), config=config)` 继续。

10. Agent 在生产部署中需要关注哪些关键指标？

- **答案：**模型调用次数/请求（<10 次）、端到端延迟（<5s）、工具调用成功率（>95%）、Token 消耗、HITL 拦截率。建议使用 LangSmith / Prometheus 监控，并部署 PII 脱敏、限流、降级等中间件确保安全。